TxSimulator

Transaction Simulation and Pre-Execution Security on BOT Chain

1. Abstract

As decentralized finance (DeFi) and smart contract interactions grow in complexity, users face increasing risks from malicious contracts, unexpected reverts, and phishing attacks. TxSimulator is a comprehensive pre-execution simulation platform built natively for BOT Chain. By leveraging RPC-based simulation methods and on-chain storage for preferences and histories, TxSimulator allows developers and retail users to preview exact state changes, gas costs, and security vectors before cryptographically signing any payload.

2. Problem Statement

Blockchain transactions are immutable. Once a user signs and broadcasts a transaction, the execution is final. Traditional wallet interfaces often display opaque hexadecimal calldata, leaving users blind to the actual state modifications (e.g., unlimited token approvals or hidden transfers) that will occur. This opacity has led to billions of dollars lost to malicious dApps and simple human error.

3. Vision

We envision a secure Web3 ecosystem where no user signs a blind transaction. TxSimulator acts as a protective layer, translating complex EVM calldata into human-readable expected outcomes, empowering users to interact with BOT Chain confidently.

4. Transaction Simulation Architecture

The architecture separates the simulation layer from the signing layer. When a user intends to execute a transaction, the payload is intercepted and routed to the TxSimulator dashboard. The engine uses BOT Chain's robust RPC endpoints (such as `eth_call` and `eth_estimateGas`) to execute the transaction context locally against the current block state without broadcasting it to the mempool.

5. Simulation Engine & Risk Detection

The core engine analyzes the response and state differences to identify key security metrics. Our risk detection suite flags:

- **Unlimited Token Approvals:** Identifies ERC20 `approve` calls with the maximum `uint256` value.
- **Suspicious Contract Addresses:** Cross-references destination addresses with known honey-pots or unverified bytecode.
- **Unexpected Token Transfers:** Flags interactions where tokens leave the user's wallet outside of the primary intended function.
- **Reverting Transactions:** Captures EVM revert reasons early to save users from wasted gas fees.

6. Technical Implementation

The platform is engineered as a zero-dependency front-end utilizing HTML5, CSS3, and Vanilla JavaScript, ensuring maximum compatibility and auditability. The `ethers.js` library facilitates RPC communication and data decoding. Advanced features include calldata decoding, function selector mapping, and historical transaction comparison.

7. Solidity Architecture & On-chain Data

TxSimulator utilizes a suite of native BOT Chain smart contracts to manage user states:

- **Profile Registry:** Stores user-defined preferences and aliases for frequent addresses.
- **Simulation Records:** Allows users to save complex calldata blueprints and successful simulation hashes for future reference or multi-sig sharing.
- **Test Suite:** Includes deployed ERC20 and vulnerable contracts to allow developers to safely test the simulator's reentrancy and access control detection.

8. BOT Chain Integration

TxSimulator is tailored for BOT Chain's high-throughput environment. The platform continuously monitors network congestion to provide accurate base fee and priority fee estimates, ensuring simulated gas costs closely match actual execution costs.

9. User Workflows

Phase	Action	Outcome
Initialization	Connect Wallet & Select Network	Validates BOT Chain connection and fetches on-chain profile.
Input	Paste Calldata / ABI	Engine decodes function selectors and parameters.
Simulation	Execute RPC Call	Retrieves estimated gas, state changes, and revert strings.
Analysis	Run Security Checks	Displays human-readable warnings and flow diagrams.

10. Conclusion & Roadmap

TxSimulator establishes a new standard for transaction safety on BOT Chain. Future iterations will include full EVM trace capabilities, automated formal verification of target contracts, and integration directly into leading browser extension wallets via snap modules.

> **Note:** While TxSimulator provides high-fidelity estimations using standard RPC calls, exact execution outcomes may vary due to block state changes between simulation and final block inclusion.